
Bowling Score Detection from Video

Using Computer Vision

Technical Report — Final Project

Farida Ehab 202201509 **Nour Osama** 202201874

Spring 2026

1. Introduction

This report documents the design, implementation, and evaluation of a computer vision pipeline built for the DSAI 352 final project. The system processes a mobile-recorded bowling video and produces a fully annotated output video that marks each pin as it falls, records the timestamp of each fall, and presents a final game summary.

The bowling setup uses a simplified variant of ten-pin bowling in which at least five pins are arranged in a chosen layout on the ground. A player rolls a standard bowling ball toward the pins across multiple throws captured within a single roughly thirty-second recording. The challenge is to detect when each individual pin transitions from standing to fallen and to maintain a stable identity for every pin throughout the entire video, despite motion blur, partial occlusion, simultaneous falls, and pins that slide out of frame.

The project satisfies the course requirement of combining a trained or fine-tuned deep learning model with OpenCV-based video processing. The report covers the full pipeline in the order: dataset collection and labeling, model selection and training, tracking and fall-detection logic, output rendering, evaluation, and known limitations.

2. Dataset Collection and Preparation

2.1 Data Collection

Training and validation data were collected by recording bowling sessions on a mobile phone. Video clips were then broken into individual frames from which representative images were extracted. The dataset covers a variety of pin arrangements, lighting conditions, and viewing angles to improve generalization.

The dataset was assembled and hosted on Roboflow under the project identifier cv-project-59lsq (workspace: nours-workspace-qpy1n). Two object classes are defined:

- pin-fallen (class index 0): a pin that has come to rest on the ground and is no longer upright.
- pin-standing (class index 1): a pin that is fully or partially upright.

2.2 Annotation and Export

Images were annotated in Roboflow using bounding boxes. The dataset was exported in YOLOv8 format, producing the directory structure and data.yaml file required for Ultralytics training. To create a validation split, the first 20 percent of training images were moved into a dedicated valid folder before training commenced.

Split	Images	Purpose
Train (80%)	~80% of collected frames	Model weight optimization
Validation (20%)	~20% of collected frames	Hyperparameter tuning and early stopping
Test	Unseen bowling recording	Final demonstration and grading

[Dataset Drive](#)

3. Model Selection and Architecture

3.1 Model Choice: YOLOv8n

The team selected YOLOv8n (nano), the lightest variant in the YOLOv8 family, as the base detection model. This choice was driven by three considerations. First, the nano variant converges quickly on small datasets, which was important given the limited amount of labeled bowling data available. Second, inference at 30 fps on a laptop CPU is feasible with the nano model, ensuring smooth video processing. Third, fine-tuning a pretrained YOLOv8 backbone satisfies the course requirement of training or applying transfer learning rather than using an off-the-shelf model without any learning step.

YOLOv8n is a single-stage anchor-free detector built on a CSP (Cross Stage Partial) backbone with a decoupled detection head. The pretrained weights are initialized from training on COCO, giving the model strong general visual features that transfer well to detecting small objects like bowling pins.

3.2 Why Not a Larger Model

YOLOv8s and YOLOv8m were considered but rejected. The dataset size is modest, and a heavier model would risk overfitting without substantially more training data. The nano variant reached satisfactory detection accuracy within the available training budget while keeping inference latency low enough to process a 30-second video in a reasonable time on Google Colab.

4. Training Procedure

4.1 Environment

Training was conducted on Google Colab using a GPU runtime. The Ultralytics library (pip install ultralytics) and Roboflow SDK (pip install roboflow) were used to download the dataset and drive training. The final trained weights are saved as best.pt.

4.2 Hyperparameters

Hyperparameter	Value	Rationale
Base model	yolov8n.pt	Pretrained COCO weights; fast convergence on small datasets
Image size	640 px	Standard YOLOv8 resolution; balances detail and speed
Batch size	16	Fits Colab GPU memory comfortably
Epochs	50	Sufficient for small dataset; early stopping guards against overfitting
Early stopping patience	10	Stops if validation mAP does not improve for 10 epochs
Confidence threshold (inference)	0.35	Tuned to minimize missed detections without excessive false positives

4.3 Training Script

The core training call is reproduced below for reference:

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt')
model.train(
    data=f'{dataset.location}/data.yaml',
    epochs=50,
    imgsz=640,
    batch=16,
    name='bowling_pin_detector',
    patience=10,
)
```

5. Inference and Tracking Pipeline

The inference pipeline reads the input video frame by frame using OpenCV, runs YOLO detection on each frame, matches detections to tracked pin identities, applies fall-detection logic, and renders annotations. The pipeline is implemented in Python and organized into the stages described below.

5.1 Initialization

The tracker waits until `INIT_STABLE_FRAMES` (3) consecutive frames each return exactly `NUM_PINS` (5) clean detections after NMS and spatial clustering. This prevents a noisy or partially visible opening frame from producing incorrect pin identities. Once the stability criterion is met, pins are sorted left-to-right by x-coordinate and assigned integer IDs 1 through 5.

5.2 Non-Maximum Suppression and Clustering

Raw YOLO detections are first filtered by a confidence threshold of 0.35. Greedy IoU-based NMS (threshold 0.45) removes duplicate detections of the same pin. A spatial clustering step then merges detections whose centroids are within 60 percent of `MAX_MATCH_DIST` (72 pixels), retaining the highest-confidence box per cluster. This two-stage filtering is necessary because YOLO sometimes produces multiple overlapping boxes for a single pin, especially when the pin is partially occluded.

5.3 Detection-to-Pin Matching

On every frame the filtered detections are matched to the currently standing tracked pins using greedy centroid-distance matching. The closest detection-to-pin pair (by Euclidean centroid distance) is assigned first, and neither the detection nor the pin is reused. Matches beyond `MAX_MATCH_DIST` (120 pixels) are rejected; this prevents a sliding or bouncing pin from incorrectly capturing the identity of a distant pin.

5.4 Fall Detection Logic

Fall confirmation uses a dual-path sliding-window approach designed to handle both fast and slow falls:

- Fast path: if the tail of the sliding window contains `CONSEC_THRESH` (2) or more consecutive `CLASS_FALLEN` detections, the fall is confirmed immediately. This catches abrupt falls caused by a direct hit.
- Slow path: if the window has accumulated `WINDOW_SIZE` (6) frames and at least `WINDOW_RATIO` (70%) of them are `CLASS_FALLEN`, the fall is confirmed. This catches pins that topple slowly or lean before falling.
- Vanished-pin rule: if a pin receives no matched detection for `GONE_CONFIRM_FRAMES` (15) consecutive frames (~0.5 s at 30 fps), it is assumed to have fallen and slid out of the model's field of view. The fall timestamp is backdated to the first missing frame.
- Last-pin auto-confirm: when exactly one pin remains standing and all others have fallen, that final pin is confirmed fallen as soon as even one `CLASS_FALLEN` detection appears in its window, preventing the video from ending with an unconfirmed fall.

5.5 Identity Maintenance

Once a pin is confirmed fallen, its status and fall timestamp are locked and no further updates are applied to it. The last known bounding box at the moment of confirmation is frozen and used for all subsequent rendering, so the annotation marker remains stable at the correct spatial position rather than jumping to the pin's post-fall resting location.

6. Output Video Rendering

The annotated output video is produced by OpenCV's `VideoWriter` at the same resolution and frame rate as the input. Each frame receives the following overlays:

- Standing pins: a green circle (radius 35 px) centered on the pin's tracked centroid, with a label 'Pin N' above it.
- Fallen pins: the green circle is replaced by a red circle, and a label reading 'Pin N fell @ T.Ts' is drawn above the circle, where T.T is the fall timestamp in seconds.

- HUD (always visible): the current count of fallen pins out of the total ('Pins down: K/5') and the elapsed video time in seconds, both displayed in yellow in the top-left corner.
- Final summary overlay (last 2 seconds): a centered cyan overlay reading 'GAME OVER', 'Total Pins Knocked: K / 5', and the total video duration.

7. Evaluation

7.1 Model Metrics

After training, the model was evaluated on the validation split using the Ultralytics val() function. The metrics reported below characterize the detector's ability to distinguish pin-fallen from pin-standing and to localize pins accurately.

Metric	Value	Notes
mAP@0.5	0.742	Mean average precision at IoU threshold 0.5
mAP@0.5:0.95	0.961	Stricter localization metric averaged over IoU thresholds
Precision (pin-fallen)	0.942	Fraction of fallen detections that are correct
Recall (pin-fallen)	0.915	Fraction of actual falls that are detected
Confidence threshold	0.35	Set empirically to balance precision and recall

Metric	All Classes	pin-fallen	pin-standing
Precision (P)	0.942	0.925	0.959
Recall (R)	0.915	0.890	0.940
mAP@50	0.961	0.948	0.974
mAP@50-95	0.742	0.685	0.799

Quantitative metrics provide an upper bound on detection quality; the practical test is whether fall timestamps in the annotated output video correspond correctly to the observed physical events in the original recording.

7.2 End-to-End Validation

End-to-end validation was performed by visually inspecting the annotated output video against the original. For each pin, the fall timestamp shown in the overlay was compared to the frame at which the pin visibly comes to rest in the input video. The dual-path fall-detection logic and vanished-pin rule were tuned iteratively based on these comparisons until all five pins received correct fall timestamps within approximately one second of the actual event.

8. Limitations and Failure Modes

Initialization: Requires a clear view of all pins at the start of the video.

Identity Swaps: Extremely rapid simultaneous falls where pins overlap may cause ID confusion.

Camera Motion: Significant handheld camera shake can break the centroid matching logic.

9. Conclusion

This project successfully integrates a fine-tuned deep learning model with a robust computer vision pipeline. By utilizing a dynamic initialization and a ratio-based fall logic, the system provides accurate bowling scores and visual annotations that meet all project specifications.

10. References

- [1] G. Jocher et al., *YOLOv8 by Ultralytics*, 2023.
- [2] Roboflow, Inc., *Computer Vision Platform*, 2024.
- [3] OpenCV Development Team, *OpenCV 4.x Documentation*, 2024.